

AADG classes:

Genome assembly with OLC approach

Norbert Dojer

26.11.2024

All you need today is python interpreter.

1 Brute-force SCS algorithm.

1. Inspect the code for the brute-force SCS algorithm.
2. Compute SCS for ['BAA', 'AAB', 'BBA', 'ABA', 'ABB', 'BBB', 'AAA', 'BAB']. Check the correctness of the result.
3. What is the time complexity of the algorithm? How will the running time change when you add a few sequences to the above list? Verify your conjecture.

2 Greedy SCS algorithm.

1. Inspect the code for the SCS algorithm with greedy selection of reads to be merged.
2. Apply the algorithm to the sequence list from the previous exercise. How will the running time change when you double the input list? Verify your conjecture.
3. Apply both greedy and brute-force SCS algorithms to the list ['ABCD', 'CDBC', 'BCDA']. Explain the difference between the results.

3 Dealing with approximate overlaps.

1. Recall the code for the DP-based approximate string matching algorithm.
2. Adapt the code to the approximate overlap finding problem.
3. Design the assembly algorithm combining approximate overlap finding with the greedy SCS algorithm. Consider the following problems:
 - How to order overlaps for the greedy algorithm?
 - How to merge approximately matching reads into one sequence?
4. Implement your algorithm.
5. Run your algorithm on the sequences from moodle with minimal overlap length set to 10 and the similarity threshold that allows 2-4 mismatches. Inspect the results: what is the total assembly length? how many contigs did you get?
6. Run the greedy SCS algorithm on the same data. Compare results.
7. Save assemblies obtained from both methods as reference .fasta files. Then map reads onto both references and inspect the results using IGB/IGV.